Лабораторная работа №5 Построение графиков

Компьютерный практикум по статистическому анализу данных

Беспутин Глеб Антонович

Содержание

1 Цель работы **5**

2 Задание **6**

3 Теоретическое введение **7**
3.1 Пакет Plots и бэкенды 7
3.2 Ряд Тейлора и оформление 7
3.3 Точки, аппроксимация, две оси 8
3.4 Кривые, поверхности, поле 8
3.5 Анимация и циклоиды 9
3.6 Погрешности и гистограммы 9

4 Выполнение лабораторной работы **10**
4.1 Построение графиков в Julia 10
4.2 Повторение примеров из раздела 5.2 13
4.3 Задания для самостоятельной работы (раздел 5.4) 48

5 Результаты **65**
5.1 Повторение примеров из раздела 5.2 65
5.2 Задания для самостоятельной работы 66

6 Выводы **69**

Список литературы **71**

Список иллюстраций

5.1 sin(x) и частичная сумма ряда Тейлора 66
5.2 Поверхность $x^2 + y^2$ 66
5.3 Четыре окна для $y = x^3 - 3x$ 68
5.4 πx и $e^{\square} \cos x$ на двух осях 68

Список таблиц

1 Цель работы

Освоить синтаксис Julia для построения графиков: бэкенды `Plots`, оформление осей и линий, точечные и трёхмерные серии, аппроксимацию, две оси ординат, полярные и параметрические кривые, поверхности, линии уровня, векторные поля, анимацию, погрешности и гистограммы.

2 Задание

1. Повторить примеры раздела 5.2 и дополнить графики подписями осей, легендой и названием.
2. Выполнить задания раздела 5.4:
   - типы графиков и стили линий для $y = \sin x$ на $[0, 2\pi]$;
   - график $y = \pi x^2 \ln x$ с заданным цветом рамки, шрифтом и частотой делений;
   - четыре окна для $y = x^3 - 3x$ в файле `figure_besputin.png`;
   - $y_1 = \pi x$ и $y_2 = e^x \cos x$ на одной оси и на двух осях;
   - экспериментальный ряд с погрешностью;
   - плоский и пространственный точечные графики;
   - анимация синусоиды;
   - анимированные гипоциклоида и эпициклоида для двух целых и двух рациональных k.

Случайные выборки порождаются с зерном `Random.seed!(42)`, поэтому прогон повторяется. Рисунки записывает `savefig` в каталог `report/image` в том виде, в каком они строятся при выполнении скрипта.

3 Теоретическое введение

3.1 Пакет Plots и бэкенды

`Plots` задаёт один набор команд, а рисует выбранная библиотека. Переключение — вызов `gr()`, `pyplot()`, `plotly()` или `unicodeplots()`. В этой работе статичные файлы png и pdf пишет GR, русские подписи на том же графике проверяются через PyPlot, интерактивная копия сохраняется бэкендом plotly (пакеты `PlotlyBase` и `PlotlyKaleido`), текстовая копия — UnicodePlots.

Функция одной переменной задаётся поэлементно. Для

$$f(x) = (3x^2 + 6x - 9)\, e^{-0,3x}$$

массив x и массив $y = f(x)$ передаются в `plot`. Подписи — атрибуты `title`, `xlabel`, `ylabel`, `label`.

3.2 Ряд Тейлора и оформление

Частичная сумма ряда синусов до степени 9:

$$\sin x \approx \sum_{n=0}^{4} \frac{(-1)^n x^{2n+1}}{(2n+1)!}.$$

`plot!` добавляет вторую серию на текущие оси. Цвет, прозрачность, толщина и стиль собираются в кортеж `line`. `fillrange` закрашивает область до задан-

ного уровня, `xticks` и `yticks` задают частоту делений, `xrotation` поворачивает подписи оси абсцисс.

3.3 Точки, аппроксимация, две оси

`scatter` и `seriestype = :scatter` рисуют маркеры. Вектор `markersize` кодирует третье значение размером точки. В трёхмерном случае в `scatter` передаются три координаты.

Полином степени 5 по точкам (x_i, y_i) ищется из условия минимума суммы квадратов отклонений. Столбцы матрицы A — это $1, x, x^2, \dots, x^5$, и коэффициенты равны $c = A \setminus y$. Оператор \ для прямоугольной матрицы даёт решение наименьших квадратов.

`twinx()` добавляет правую ось ординат. Она нужна, когда размахи двух рядов различаются на порядок: общая ось оставляет мелкий ряд почти горизонтальной полосой.

3.4 Кривые, поверхности, поле

В полярных координатах `proj = :polar`, радиус считается как $r(\theta)$. Параметрическая кривая на плоскости задаётся двумя функциями параметра, в пространстве — тремя массивами. `surface` и `linetype = :wireframe` строят $f(x, y)$. `contour` проводит линии уровня, `fill = true` заливает промежутки между ними.

Градиент $h(x, y) = x^3 - 3x + y^2$ равен $(3x^2 - 3,\ 2y)$. `quiver!` ставит эти векторы на сетку. Деление на 25 удерживает длину стрелки в масштабе рисунка.

3.5 Анимация и циклоиды

`@animate` собирает кадры, `gif` записывает их в один файл. Кадр синусоиды — график на всё более длинном отрезке аргумента.

Гипоциклоида — след точки окружности радиуса r, катящейся внутри окружности радиуса $R = kr$:

$$x = r(k-1)\cos t + r\cos((k-1)t), \quad y = r(k-1)\sin t - r\sin((k-1)t).$$

Эпициклоида — та же точка, окружность катится снаружи:

$$x = r(k+1)\cos t - r\cos((k+1)t), \quad y = r(k+1)\sin t - r\sin((k+1)t).$$

Для $k = p/q$ в несократимых целых кривая замыкается за q оборотов, поэтому параметр доводится до $2\pi q$.

3.6 Погрешности и гистограммы

Для среднего n нормальных величин с разбросом σ полуширина интервала берётся $1{,}96\,\sigma/\sqrt{n}$. На графике это `yerr`, `xerr` или лента `ribbon`. Асимметричный интервал задаётся парой векторов.

`Distributions` даёт `Normal(□, □)` и выборку `rand`. `histogram` строит частоты. Несколько окон в одной фигуре задаёт `layout`: кортеж `(N, M)`, целое число, `grid` с долями высоты или макрос `@layout`.

4 Выполнение лабораторной работы

Скрипт project/scripts/lab05-literate.jl повторяет раздел 5.2 и решает задания раздела 5.4. Команда julia --project=. scripts/tangle.jl scripts/lab05-literate.jl собирает из него чистый скрипт, документ Quarto и блокнот Jupyter Lab. Ниже приведён этот документ: пояснения и листинг.

4.1 Построение графиков в Julia

Пакеты Plots, PyPlot, UnicodePlots, PlotlyBase, PlotlyKaleido, Distributions и Statistics стоят в окружении проекта: их ставит add_packages.jl. В методичке те же пакеты добавляют через Pkg.add при первом запуске.

Случайные выборки фиксируются зерном Random.seed!(42). Рисунки пишутся в report/image рядом с отчётом. GKSwstype=nul и MPLBACKEND=Agg отключают окно графика: файл строится тот же, что открылся бы на экране.

```
ENV["GKSwstype"] = "nul"
ENV["MPLBACKEND"] = "Agg"

using DrWatson
@quickactivate "project"

using Plots
using Plots.PlotMeasures
```

```
using Random
using Statistics
using Distributions
using LinearAlgebra
import PyPlot

Random.seed!(42)

FIGDIR = projectname() == "project" ?
         joinpath(dirname(projectdir()), "report", "image") :
         joinpath(pwd(), "image")
mkpath(FIGDIR)

function saveplot(name)
    path = joinpath(FIGDIR, name)
    savefig(path)
    println("рисунок: ", name)
    return path
end

function save_unicode(plt, name)
    io = IOBuffer()
    show(io, MIME"text/plain"(), plt)
    text = String(take!(io))
    println(text)
    txtpath = joinpath(FIGDIR, replace(name, ".png" => ".txt"))
    write(txtpath, text)
    pngpath = joinpath(FIGDIR, name)
```

```
    lines = split(text, '\n')
    width = maximum(length, lines)
    fig_w = max(6.5, width * 0.092)
    fig_h = max(2.4, length(lines) * 0.22)
    prop = PyPlot.matplotlib.font_manager.FontProperties(
        fname = "/usr/share/fonts/truetype/dejavu/DejaVuSansMono.ttf",
        size = 11,
    )
    fig = PyPlot.figure(figsize = (fig_w, fig_h), dpi = 140)
    fig.patch.set_facecolor("white")
    ax = fig.add_axes((0.01, 0.02, 0.98, 0.96))
    ax.set_axis_off()
    ax.text(0.0, 1.0, text, fontproperties = prop, va = "top", ha = "left")
    fig.savefig(pngpath, facecolor = "white")
    PyPlot.close(fig)
    println("рисунок: ", name)
    return text
end

gr()
default(
    size = (720, 480),
    show = false,
    guidefont = font("DejaVu Sans", 11),
    titlefont = font("DejaVu Sans", 13),
    tickfont = font("DejaVu Sans", 9),
    legendfont = font("DejaVu Sans", 9),
)
```

```
if isdefined(Main, :IJulia) && IJulia.inited
    IJulia.display_dict(::Plots.Plot) = Dict{String,String}()
end
```

```
┌ Warning: DrWatson could not find find a project file by recursively checking given `dir
│  (given dir: /home/glebq)
└ @ DrWatson ~/.julia/packages/DrWatson/2QF5p/src/project_setup.jl:84
```

4.2 Повторение примеров из раздела 5.2

4.2.1 Бэкенды Plots (раздел 5.2.1)

Одна и та же кривая

$$f(x) = (3x^2 + 6x - 9)\, e^{-0{,}3x}$$

строится массивами x и y. Plots меняет библиотеку вызовом `gr()`, `pyplot()`, `plotly()` и `unicodeplots()`.

```
f(x) = (3 .* x .^ 2 .+ 6 .* x .- 9) .* exp.(-0.3 .* x)
x = collect(range(-5, 10, length = 151))
y = f(x)

gr()
plot(
    x, y,
    title = "A simple curve",
    xlabel = "Variable x",
    ylabel = "Variable y",
    label = "f(x)",
```

```
    color = "blue",
    legend = :topright,
)
saveplot("fig-5-1-gr.png")

pyplot()
plot(
    x, y,
    title = "Простая кривая",
    xlabel = "Переменная x",
    ylabel = "Переменная y",
    label = "f(x)",
    color = "blue",
    legend = :topright,
)
saveplot("fig-5-2-pyplot.png")

plotly()
plot(
    x, y,
    title = "Простая кривая, plotly",
    xlabel = "Переменная x",
    ylabel = "Переменная y",
    label = "f(x)",
    color = "blue",
    legend = :topright,
)
savefig(joinpath(FIGDIR, "fig-plotly-simple.html"))
```

```
println("рисунок: fig-plotly-simple.html")
try
    saveplot("fig-plotly-simple.png")
catch err
    println("plotly png не записан: ", err)
end

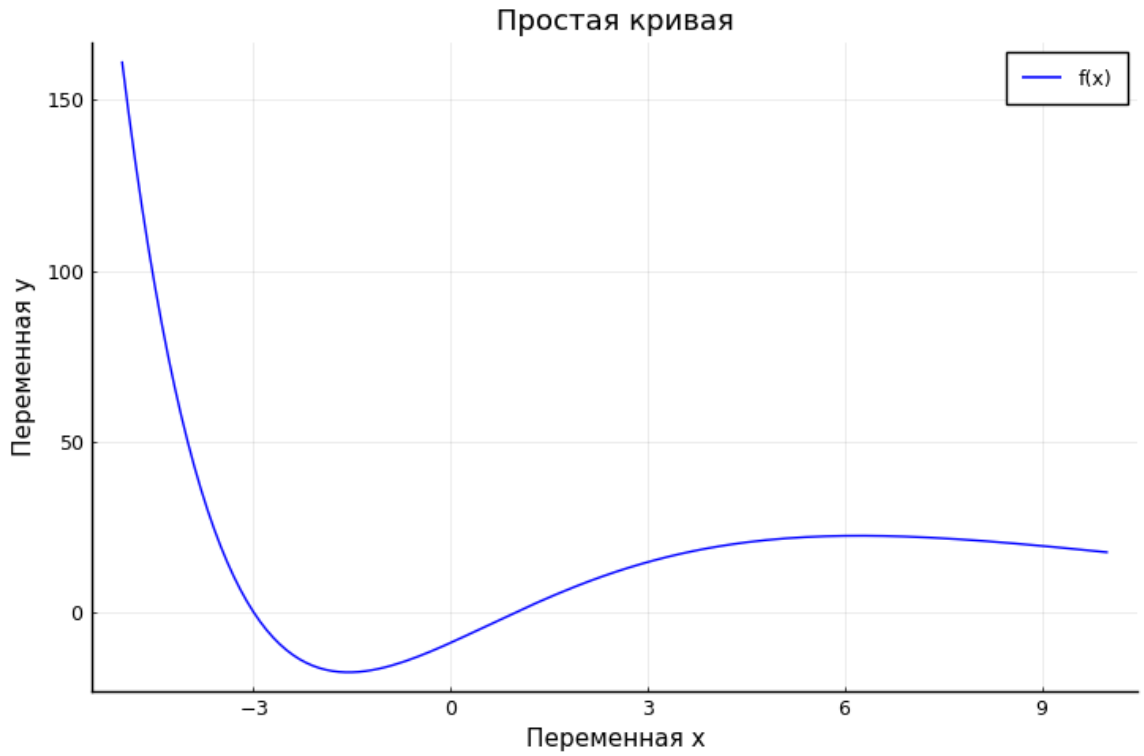
unicodeplots()
plt_unicode = plot(
    x, y,
    title = "f(x), unicodeplots",
    xlabel = "x",
    ylabel = "y",
    label = "f(x)",
)
save_unicode(plt_unicode, "fig-unicode-simple.png")

gr()
```

```
рисунок: fig-5-1-gr.png
рисунок: fig-5-2-pyplot.png
рисунок: fig-plotly-simple.html
рисунок: fig-plotly-simple.png
                    f(x), unicodeplots
        +----------------------------------------+
  166.71| ,           |                          |f(x)
        | \           |                          |
        | |           |                          |
        | |           |                          |
```

```
        | ”,          |                          |
        |  \          |                          |
        |  .          |                          |
       y|  ”,         |                          |
        |   \         |                          |
        |   ,         |                          |
        |    .        |                          |
        |    \.       |       .__------------._. |
        |     \       |  ._-*”`                  |
        |”””””””|”””””””]}T”””””””””””””””””””””””””””|
-23.0172|       '----”|                          |
        +-----------------------------------------+
         -5.45              x               10.45
```

рисунок: fig-unicode-simple.png



```
Plots.GRBackend()
```

4.2.2 Оформление графика и ряд Тейлора (раздел 5.2.2)

Ряд Тейлора для синуса, пять слагаемых (степень 9):

$$\sin x \approx \sum_{n=0}^{4} \frac{(-1)^n x^{2n+1}}{(2n+1)!}.$$

```
sin_theor(x) = sin(x)
sin_taylor(x) = sum((-1)^i * x^(2 * i + 1) / factorial(2 * i + 1) for i in 0:4)

plot(
    sin_theor, -2π, 2π,
    title = "График функции sin(x)",
    xlabel = "x",
    ylabel = "sin(x)",
    label = "sin(x)",
    legend = :topright,
    lw = 2,
)
saveplot("fig-5-3-sin.png")

plot(
    sin_taylor, -2π, 2π,
    title = "Ряд Тейлора для sin(x)",
    xlabel = "x",
    ylabel = "значение",
    label = "ряд Тейлора, n = 0…4",
    legend = :topright,
```

```
    lw = 2,
    ylims = (-2, 2),
)
saveplot("fig-5-4-taylor.png")

plot(
    sin_theor, -2π, 2π,
    title = "sin(x) и ряд Тейлора",
    xlabel = "x",
    ylabel = "значение",
    label = "sin(x)",
    legend = :topright,
    lw = 2,
    ylims = (-2, 2),
)
plot!(sin_taylor, -2π, 2π, label = "ряд Тейлора", lw = 2)
saveplot("fig-5-5-both.png")

xs_t = range(-5, 5, length = 600)
plot(
    xs_t, sin_taylor.(xs_t),
    label = "sin(x), разложение в ряд Тейлора",
    line = (:blue, 0.3, 6, :solid),
    size = (800, 500),
    xticks = -5:1:5,
    yticks = -1:0.5:1,
    xtickfont = font(11, "Times"),
    ytickfont = font(11, "Times"),
```

```
    ylabel = "y",
    xlabel = "x",
    title = "Разложение в ряд Тейлора",
    xrotation = rad2deg(π / 4),
    fillrange = 0,
    fillalpha = 0.5,
    fillcolor = :lightgoldenrod,
    background_color = :ivory,
    ylims = (-1.2, 1.2),
    xlims = (-5, 5),
    legend = :topright,
    bottom_margin = 14mm,
    left_margin = 8mm,
)
plot!(
    xs_t, sin.(xs_t),
    label = "sin(x), теоретическое значение",
    line = (:black, 1.0, 2, :dash),
)
saveplot("fig-5-6-styled.png")
savefig(joinpath(FIGDIR, "taylor.png"))
savefig(joinpath(FIGDIR, "taylor.pdf"))
println("рисунок: taylor.png")
println("рисунок: taylor.pdf")
```

```
рисунок: fig-5-3-sin.png
рисунок: fig-5-4-taylor.png
рисунок: fig-5-5-both.png
рисунок: fig-5-6-styled.png
```

```
рисунок: taylor.png
рисунок: taylor.pdf
```

4.2.3 Точечные графики (раздел 5.2.3)

К примерам методички добавлены подписи осей, легенда и заголовок.

```
Random.seed!(42)
xs_sc = range(1, 10, length = 10)
ys_sc = rand(10)
println("Точки раздела 5.2.3, y = ", ys_sc)
plot(
    xs_sc, ys_sc,
    seriestype = :scatter,
    title = "Точечный график",
    xlabel = "номер точки",
    ylabel = "значение",
    label = "10 случайных значений",
    legend = :topright,
    markersize = 6,
)
saveplot("fig-5-7-scatter.png")

Random.seed!(42)
n = 50
xs_sc = rand(n)
ys_sc = rand(n)
ms = rand(n) .* 30
scatter(
```

```
    xs_sc, ys_sc,
    markersize = ms,
    title = "Размер точки кодирует третье значение",
    xlabel = "x",
    ylabel = "y",
    label = "50 точек",
    legend = :topright,
    color = :teal,
    alpha = 0.75,
)
saveplot("fig-5-8-scatter-size.png")

Random.seed!(42)
xs_sc = rand(n)
ys_sc = rand(n)
zs_sc = rand(n)
ms = rand(n) .* 30
scatter(
    xs_sc, ys_sc, zs_sc,
    markersize = ms,
    title = "Точки в пространстве",
    xlabel = "x",
    ylabel = "y",
    zlabel = "z",
    label = "50 точек",
    legend = :topright,
    color = :purple,
    alpha = 0.8,
```

```
    size = (720, 560),
)
saveplot("fig-5-9-scatter3d.png")
```

```
Точки раздела 5.2.3, y = [0.1735745757945074, 0.32166161915780656, 0.25858546995315457,
рисунок: fig-5-7-scatter.png
рисунок: fig-5-8-scatter-size.png
рисунок: fig-5-9-scatter3d.png
```

```
"/home/glebq/2026-1--study--computer-practice/labs/lab05/report/image/fig-
5-9-scatter3d.png"
```

4.2.4 Аппроксимация полиномом (раздел 5.2.4)

Точки лежат около e^{1+x} со сдвигом 4000 ε, $\varepsilon \sim N(0,1)$. Коэффициенты полинома 5-й степени решают систему $Ac = y$ в смысле наименьших квадратов: оператор \.

```
Random.seed!(42)
x = collect(0:0.01:9.99)
y = exp.(ones(1000) .+ x) .+ 4000 .* randn(1000)
scatter(
    x, y,
    markersize = 3,
    alpha = 0.35,
    label = "данные",
    xlabel = "x",
    ylabel = "y",
    title = "Экспонента со случайным сдвигом",
    legend = :topleft,
```

```
    color = :gray,
)
saveplot("fig-5-10-data.png")

A = [ones(1000) x x .^ 2 x .^ 3 x .^ 4 x .^ 5]
c = A \ y
fit_y = c[1] .* ones(1000) .+ c[2] .* x .+ c[3] .* x .^ 2 .+ c[4] .* x .^ 3 .+ c[5] .
plot!(x, fit_y, linewidth = 3, color = :red, label = "полином 5-й степени")
saveplot("fig-5-11-fit.png")
println("коэффициенты c = ", c)
println("||y - F(x)|| = ", norm(y - fit_y))
```

```
рисунок: fig-5-10-data.png
рисунок: fig-5-11-fit.png
коэффициенты c = [-587.6603575708605, 2329.563997590053, -2518.1707196476273, 935.63650
141.43850899540374, 7.655559669109303]
||y - F(x)|| = 125162.66781630563
```

4.2.5 Две оси ординат (раздел 5.2.5)

```
Random.seed!(42)
plot(
    randn(100),
    ylabel = "y1",
    xlabel = "номер отсчёта",
    label = "траектория 1",
    legend = :topright,
    grid = :off,
```

```
    title = "Одна траектория",
    color = :steelblue,
)
saveplot("fig-5-12-traj.png")

plot!(
    twinx(), randn(100) .* 10,
    color = :red,
    ylabel = "y2",
    label = "траектория 2",
    legend = :bottomright,
    grid = :off,
    framestyle = :box,
)
saveplot("fig-5-13-twin.png")
```

```
рисунок: fig-5-12-traj.png
рисунок: fig-5-13-twin.png

"/home/glebq/2026-1--study--computer-practice/labs/lab05/report/image/fig-
5-13-twin.png"
```

4.2.6 Полярные координаты (раздел 5.2.6)

$$r(\theta) = 1 + \cos\theta \, \sin^2 \theta, \qquad \theta \in [0, 2\pi].$$

```
r□(□) = 1 + cos(□) * sin(□)^2
□ = range(0, stop = 2π, length = 200)
plot(
```

```
    □, r□.(□),
    proj = :polar,
    lims = (0, 1.5),
    title = "График в полярных координатах",
    label = "r(□) = 1 + cos(□) sin²(□)",
    legend = :topright,
)
saveplot("fig-5-14-polar.png")
```

```
рисунок: fig-5-14-polar.png
```

```
"/home/glebq/2026-1--study--computer-practice/labs/lab05/report/image/fig-5-14-polar.png"
```

4.2.7 Параметрические кривые (раздел 5.2.7)

На плоскости: $x(t) = \sin t$, $y(t) = \sin 2t$, $0 \le t \le 2\pi$.

В пространстве: $x(t) = \cos t$, $y(t) = \sin t$, $z(t) = \sin 5t$. Третья координата — z, как в постановке примера.

```
x_param(t) = sin(t)
y_param(t) = sin(2t)
plot(
    x_param, y_param, 0, 2π,
    fill = (0, :orange),
    xlabel = "x",
    ylabel = "y",
    title = "Параметрическая кривая на плоскости",
    label = "x = sin t, y = sin 2t",
    legend = :topright,
```

```
    aspect_ratio = :equal,
)
saveplot("fig-5-15-parametric.png")

t = range(0, stop = 10, length = 1000)
x = cos.(t)
y = sin.(t)
z = sin.(5t)
plot(
    x, y, z,
    xlabel = "x",
    ylabel = "y",
    zlabel = "z",
    title = "Параметрическая кривая в пространстве",
    label = "x = cos t, y = sin t, z = sin 5t",
    legend = :topright,
    lw = 2,
    size = (720, 560),
)
saveplot("fig-5-16-parametric3d.png")
```

рисунок: fig-5-15-parametric.png

рисунок: fig-5-16-parametric3d.png

"/home/glebq/2026-1--study--computer-practice/labs/lab05/report/image/fig-5-16-parametric3d.png"

4.2.8 Поверхности (раздел 5.2.8)

$$f(x, y) = x^2 + y^2.$$

```
fsurf(x, y) = x^2 + y^2
xs_sf = -10:10
surface(
    xs_sf, xs_sf, fsurf,
    xlabel = "x",
    ylabel = "y",
    zlabel = "f(x, y)",
    title = "surface: x² + y²",
    colorbar_title = "f",
)
saveplot("fig-5-17-surface.png")

plot(
    xs_sf, xs_sf, fsurf,
    linetype = :wireframe,
    xlabel = "x",
    ylabel = "y",
    zlabel = "f(x, y)",
    title = "Каркас x² + y²",
)
saveplot("fig-5-18-wire.png")

xs_sf = -10:0.1:10
plot(
```

```
    xs_sf, xs_sf, fsurf,
    linetype = :surface,
    xlabel = "x",
    ylabel = "y",
    zlabel = "f(x, y)",
    title = "Сглаженная поверхность",
)
saveplot("fig-5-19-smooth.png")

xs_cam = range(-2, stop = 2, length = 100)
ys_cam = range(sqrt(2), stop = 2, length = 100)
fcam(x, y) = x * y - x - y + 1
plot(
    xs_cam, ys_cam, fcam,
    linetype = :surface,
    c = cgrad([:red, :blue]),
    camera = (-30, 30),
    xlabel = "x",
    ylabel = "y",
    zlabel = "f(x, y)",
    title = "Угол зрения (−30°, 30°)",
)
saveplot("fig-5-20-camera.png")
```

рисунок: fig-5-17-surface.png

рисунок: fig-5-18-wire.png

рисунок: fig-5-19-smooth.png

рисунок: fig-5-20-camera.png

"/home/glebq/2026-1--study--computer-practice/labs/lab05/report/image/fig-

5-20-camera.png”

4.2.9 Линии уровня (раздел 5.2.9)

$$g(x, y) = (3x + y^2)\,|\sin x + \cos y|.$$

```
xs_g = 1:0.5:20
ys_g = 1:0.5:10
g(x, y) = (3x + y^2) * abs(sin(x) + cos(y))
plot(
    xs_g, ys_g, g,
    linetype = :surface,
    xlabel = ”x”,
    ylabel = ”y”,
    zlabel = ”g(x, y)”,
    title = ”Поверхность g(x, y)”,
)
saveplot(”fig-5-21-g.png”)

contour(
    xs_g, ys_g, g,
    xlabel = ”x”,
    ylabel = ”y”,
    title = ”Линии уровня g(x, y)”,
    colorbar_title = ”g”,
)
saveplot(”fig-5-22-contour.png”)
```

```
contour(
    xs_g, ys_g, g,
    fill = true,
    xlabel = "x",
    ylabel = "y",
    title = "Линии уровня с заливкой",
    colorbar_title = "g",
)
saveplot("fig-5-23-contour-fill.png")
```

```
рисунок: fig-5-21-g.png
рисунок: fig-5-22-contour.png
рисунок: fig-5-23-contour-fill.png

"/home/glebq/2026-1--study--computer-practice/labs/lab05/report/image/fig-5-23-contour-fill.png"
```

4.2.10 Векторное поле (раздел 5.2.10)

$$h(x, y) = x^3 - 3x + y^2, \qquad \nabla h = (3x^2 - 3,\ 2y).$$

Стрелки делятся на 25, иначе они закрывают линии уровня. Сетка стрелок — 12×12 на квадрате $[-2, 2]^2$.

```
Xh = range(-2, stop = 2, length = 100)
Yh = range(-2, stop = 2, length = 100)
h(x, y) = x^3 - 3x + y^2
plot(
    Xh, Yh, h,
```

```
    linetype = :surface,
    xlabel = "x",
    ylabel = "y",
    zlabel = "h(x, y)",
    title = "Поверхность h(x, y)",
)
saveplot("fig-5-24-h.png")

contour(
    Xh, Yh, h,
    xlabel = "x",
    ylabel = "y",
    title = "Линии уровня h(x, y)",
    colorbar_title = "h",
)
saveplot("fig-5-25-h-contour.png")

xq = range(-2, stop = 2, length = 12)
yq = range(-2, stop = 2, length = 12)
xs_q = vec([x for x in xq, _ in yq])
ys_q = vec([y for _ in xq, y in yq])
us_q = vec([(3x^2 - 3) / 25 for x in xq, _ in yq])
vs_q = vec([2y / 25 for _ in xq, y in yq])
quiver!(xs_q, ys_q, quiver = (us_q, vs_q), color = :blue)
saveplot("fig-5-26-quiver.png")
xlims!(-2, 2)
ylims!(-2, 2)
title!("Векторное поле, область [-2, 2]")
```

```
saveplot("fig-5-27-quiver-lims.png")
```

```
рисунок: fig-5-24-h.png
рисунок: fig-5-25-h-contour.png
рисунок: fig-5-26-quiver.png
рисунок: fig-5-27-quiver-lims.png
```

```
"/home/glebq/2026-1--study--computer-practice/labs/lab05/report/image/fig-5-27-quiver-lims.png"
```

4.2.11 Анимация поверхности (раздел 5.2.11.1)

Кадр i — поверхность $\sin(x + 10\sin i) + \cos y$. Методичка включает `pyplot()` перед `@gif`. Здесь кадры считает GR: макрос `@animate` и запись `gif` те же, окно не требуется.

```
i = 0.0
Xs = Ys = range(-5, stop = 5, length = 40)
surface(
    Xs, Ys, (x, y) -> sin(x + 10sin(i)) + cos(y),
    xlabel = "x",
    ylabel = "y",
    zlabel = "z",
    title = "Поверхность при i = 0",
    legend = false,
)
saveplot("fig-5-28-static.png")

anim = @animate for i in range(0, stop = 2π, length = 48)
    surface(
```

```
        Xs, Ys, (x, y) -> sin(x + 10sin(i)) + cos(y),
        xlabel = "x",
        ylabel = "y",
        zlabel = "z",
        title = "Анимация поверхности",
        legend = false,
        colorbar = false,
        clims = (-2.5, 2.5),
    )
end
gif(anim, joinpath(FIGDIR, "fig-5-29-surface.gif"), fps = 12)
println("рисунок: fig-5-29-surface.gif")

i = π / 2
surface(
    Xs, Ys, (x, y) -> sin(x + 10sin(i)) + cos(y),
    xlabel = "x",
    ylabel = "y",
    zlabel = "z",
    title = "Кадр анимации, i = π/2",
    legend = false,
    clims = (-2.5, 2.5),
)
saveplot("fig-5-29-frame.png")
```

```
рисунок: fig-5-28-static.png

[ Info: Saved animation to /home/glebq/2026-1--study--computer-practice/labs/lab05/report/image/fig-5-29-surface.gif
```

рисунок: fig-5-29-surface.gif

рисунок: fig-5-29-frame.png

"/home/glebq/2026-1--study--computer-practice/labs/lab05/report/image/fig-5-29-frame.png"

4.2.12 Гипоциклоида (раздел 5.2.11.2)

Точка окружности радиуса r, катящейся внутри окружности радиуса $R = kr$:

$$x = r(k-1)\left(\cos t + \frac{\cos((k-1)t)}{k-1}\right), \quad y = r(k-1)\left(\sin t - \frac{\sin((k-1)t)}{k-1}\right).$$

Ниже $k = 3, r = 1$: дельтоида. Сначала большая окружность, затем половина кривой, малая окружность и её радиус. В конце те же кадры собираются в gif.

```
function textbook_hypocycloid()
    r = 1.0
    k = 3
    n = 100
     = collect(0:(2π / 100):(2π + 2π / 100))
    X = r * k * cos.()
    Y = r * k * sin.()
    lim = 4

    plt = plot(
        xlim = (-lim, lim), ylim = (-lim, lim),
        aspect_ratio = 1, legend = false, framestyle = :origin,
        xlabel = "x", ylabel = "y",
        title = "Большая окружность, k = 3",
```

```
    size = (640, 640),
)
plot!(plt, X, Y, color = :blue)
saveplot("fig-5-30-circle.png")

i = 50
t = θ[1:i]
x = r * (k - 1) * cos.(t) .+ r * cos.((k - 1) .* t)
y = r * (k - 1) * sin.(t) .- r * sin.((k - 1) .* t)
plot!(x, y, color = :red, lw = 2)
title!("Половина пути гипоциклоиды")
saveplot("fig-5-31-half.png")

xc = r * (k - 1) * cos(t[end]) .+ r * cos.(θ)
yc = r * (k - 1) * sin(t[end]) .+ r * sin.(θ)
plot!(xc, yc, color = :black)
title!("Малая окружность")
saveplot("fig-5-32-small.png")

xl = [r * (k - 1) * cos(t[end]), x[end]]
yl = [r * (k - 1) * sin(t[end]), y[end]]
plot!(xl, yl, marker = :circle, markersize = 4, color = :black)
scatter!([x[end]], [y[end]], color = :red, markerstrokecolor = :red, markersize =
title!("Радиус малой окружности")
saveplot("fig-5-33-radius.png")

anim = @animate for i in 1:n
    t = θ[1:i]
```

```
        x = r * (k - 1) * cos.(t) .+ r * cos.((k - 1) .* t)
        y = r * (k - 1) * sin.(t) .- r * sin.((k - 1) .* t)
        plt = plot(
            xlim = (-lim, lim), ylim = (-lim, lim),
            aspect_ratio = 1, legend = false, framestyle = :origin,
            xlabel = "x", ylabel = "y",
            title = "Гипоциклоида, k = 3",
            size = (640, 640),
        )
        plot!(plt, X, Y, color = :blue)
        plot!(x, y, color = :red, lw = 2)
        xc = r * (k - 1) * cos(t[end]) .+ r * cos.(□)
        yc = r * (k - 1) * sin(t[end]) .+ r * sin.(□)
        plot!(xc, yc, color = :black)
        xl = [r * (k - 1) * cos(t[end]), x[end]]
        yl = [r * (k - 1) * sin(t[end]), y[end]]
        plot!(xl, yl, marker = :circle, markersize = 4, color = :black)
        scatter!([x[end]], [y[end]], color = :red, markerstrokecolor = :red, markersi
    end
    gif(anim, joinpath(FIGDIR, "hypocycloid.gif"), fps = 15)
    println("рисунок: hypocycloid.gif")
end

textbook_hypocycloid()
```

```
рисунок: fig-5-30-circle.png
рисунок: fig-5-31-half.png
рисунок: fig-5-32-small.png
рисунок: fig-5-33-radius.png
```

рисунок: hypocycloid.gif

[Info: Saved animation to /home/glebq/2026-1--study--computer-practice/labs/lab05/report/image/hypocycloid.gif

4.2.13 Погрешности (раздел 5.2.12)

Для каждого σ из набора $\{1, 1/2, \ldots, 1/32\}$ берётся среднее $n = 10$ нормальных величин. Полуширина интервала — $1{,}96\,\sigma/\sqrt{n}$.

```
sds = [1, 1 / 2, 1 / 4, 1 / 8, 1 / 16, 1 / 32]
n = 10
Random.seed!(42)
y = [mean(sd * randn(n)) for sd in sds]
errs = 1.96 .* sds ./ sqrt(n)
println("средние y = ", y)
println("погрешности = ", errs)

plot(
    y,
    ylims = (-1, 1),
    xlabel = "номер серии",
    ylabel = "среднее",
    title = "Средние без погрешности",
    label = "среднее",
    legend = :topright,
    marker = :circle,
)
saveplot("fig-5-35-raw.png")
```

```
plot(
    y,
    ylims = (-1, 1),
    yerr = errs,
    xlabel = "номер серии",
    ylabel = "среднее",
    title = "Средние с погрешностью",
    label = "среднее ± 1.96 □/√n",
    legend = :topright,
    marker = :circle,
)
saveplot("fig-5-36-err.png")

plot(
    y, 1:length(y),
    xerr = errs,
    marker = stroke(3, :orange),
    xlabel = "среднее",
    ylabel = "номер серии",
    title = "Погрешность по оси абсцисс",
    label = "серия",
    legend = :topright,
)
saveplot("fig-5-37-horizontal.png")

plot(
    y,
    ribbon = errs,
```

```
    fillcolor = :cyan,
    xlabel = "номер серии",
    ylabel = "среднее",
    title = "Лента погрешности",
    label = "среднее",
    legend = :topright,
    lw = 2,
)
saveplot("fig-5-38-ribbon.png")

Random.seed!(42)
n = 10
samples_x = [(rand() + 1) .* randn(n) .+ 2i for i in 1:5]
samples_y = [(rand() + 1) .* randn(n) .+ i for i in 1:5]
spread(v) = 1.96 * std(v) / sqrt(n)
xerr = map(spread, samples_x)
yerr = map(spread, samples_y)
xm = map(mean, samples_x)
ym = map(mean, samples_y)
println("xm = ", xm)
println("ym = ", ym)
println("xerr = ", xerr)
println("yerr = ", yerr)

plot(
    xm, ym,
    xerr = xerr,
    yerr = yerr,
```

```
    marker = stroke(2, :orange),
    xlabel = "x",
    ylabel = "y",
    title = "Погрешности по двум осям",
    label = "среднее пяти серий",
    legend = :topleft,
)
saveplot("fig-5-39-xyerr.png")

plot(
    xm, ym,
    xerr = (0.5 .* xerr, 2 .* xerr),
    yerr = (0.5 .* yerr, 2 .* yerr),
    marker = stroke(2, :orange),
    xlabel = "x",
    ylabel = "y",
    title = "Асимметричные погрешности",
    label = "0.5 погрешности и 2 погрешности",
    legend = :topleft,
)
saveplot("fig-5-40-asym.png")
```

```
средние y = [-0.22910160687398143, -0.05354184613671855, 0.1465652664999098, 0.01165631
0.015267637159190476, 0.00019066549941629333]
погрешности = [0.6198064213930023, 0.3099032106965012, 0.1549516053482506, 0.077475802
рисунок: fig-5-35-raw.png
рисунок: fig-5-36-err.png
рисунок: fig-5-37-horizontal.png
рисунок: fig-5-38-ribbon.png
```

```
xm = [1.447543240895107, 4.27660972145277, 6.689293409907592, 8.229174483536573, 9.7547
ym = [0.8413637717022411, 1.9133084282764572, 2.8247324161603387, 3.6594323330346135, 5
xerr = [0.800392684518261, 0.5416913438846163, 0.5790604877619879, 0.7447022487053957,
yerr = [1.147428026235027, 0.9018676081306148, 0.7526342092704283, 0.7574703799610129,
рисунок: fig-5-39-xyerr.png
рисунок: fig-5-40-asym.png

"/home/glebq/2026-1--study--computer-practice/labs/lab05/report/image/fig-
5-40-asym.png"
```

4.2.14 Гистограммы и Distributions (раздел 5.2.13)

```
Random.seed!(42)
ages = rand(15:55, 1000)
println("равномерные возрасты: min = ", minimum(ages), ", max = ", maximum(ages), ",
histogram(
    ages,
    xlabel = "возраст, лет",
    ylabel = "количество",
    title = "Выборка возрастов 15…55",
    label = "частота",
    legend = :topright,
    color = :steelblue,
    bins = 20,
)
saveplot("fig-5-41-uniform.png")

d = Normal(35.0, 10.0)
ages = rand(d, 1000)
```

```
println("N(35, 10): mean = ", mean(ages), ", std = ", std(ages))
histogram(
    ages,
    bins = 30,
    label = "Распределение по возрастам (года)",
    xlabel = "Возраст (лет)",
    ylabel = "Количество",
    title = "Нормальное распределение N(35, 10²)",
    color = :darkorange,
    legend = :topright,
)
saveplot("fig-5-42-normal.png")

Random.seed!(42)
d1 = Normal(10.0, 5.0)
d2 = Normal(35.0, 10.0)
d3 = Normal(60.0, 5.0)
N = 1000
ages = Float64[]
ages = append!(ages, rand(d1, Int64(ceil(N / 2))))
ages = append!(ages, rand(d2, N))
ages = append!(ages, rand(d3, Int64(ceil(N / 3))))
println("смесь: n = ", length(ages), ", mean = ", mean(ages))
histogram(
    ages,
    bins = 50,
    label = "Распределение по возрастам (года)",
    xlabel = "Возраст (лет)",
```

```
    ylabel = "Количество",
    title = "Смесь трёх нормальных распределений",
    color = :seagreen,
    legend = :topright,
)
saveplot("fig-5-43-mixture.png")

plotly()
histogram(
    ages,
    bins = 50,
    label = "Распределение по возрастам (года)",
    xlabel = "Возраст (лет)",
    ylabel = "Количество",
    title = "Смесь, бэкенд plotly",
)
savefig(joinpath(FIGDIR, "fig-5-43-mixture-plotly.html"))
println("рисунок: fig-5-43-mixture-plotly.html")
try
    saveplot("fig-5-43-mixture-plotly.png")
catch err
    println("plotly png смеси не записан: ", err)
end
gr()
```

```
равномерные возрасты: min = 15, max = 55, mean = 35.096
рисунок: fig-5-41-uniform.png
N(35, 10): mean = 34.72471429902291, std = 10.339891179211318
рисунок: fig-5-42-normal.png
```

```
смесь: n = 1834, mean = 32.55059392098629
рисунок: fig-5-43-mixture.png
рисунок: fig-5-43-mixture-plotly.html
рисунок: fig-5-43-mixture-plotly.png

Plots.GRBackend()
```

4.2.15 Подграфики (раздел 5.2.14)

```
Random.seed!(42)
x = range(-2, 2, length = 10)
y = rand(10, 4)
println("матрица подграфиков 10×4, первый столбец = ", y[:, 1])

plot(
    x, y,
    layout = (4, 1),
    xlabel = "x",
    ylabel = "y",
    title = ["ряд 1" "ряд 2" "ряд 3" "ряд 4"],
    legend = false,
    size = (700, 800),
)
saveplot("fig-5-44-rows.png")

plot(
    x, y,
    layout = 4,
    xlabel = "x",
```

```
    ylabel = "y",
    legend = false,
    size = (700, 700),
    title = ["ряд 1" "ряд 2" "ряд 3" "ряд 4"],
)
saveplot("fig-5-45-grid.png")
```

```
матрица подграфиков 10×4, первый столбец = [0.1735745757945074, 0.32166161915780656, 0.2
рисунок: fig-5-44-rows.png
рисунок: fig-5-45-grid.png

"/home/glebq/2026-1--study--computer-practice/labs/lab05/report/image/fig-
5-45-grid.png"
```

В методичке доли высоты равны 0.20, 0.30, 0.40 и 0.15. Их сумма равна 1.05. Эта версия Plots принимает только набор, который в сумме даёт 1, поэтому доли делятся на свою сумму. Отношение высот остаётся тем же.

```
heights_raw = [0.2, 0.3, 0.4, 0.15]
println("сумма долей из методички = ", sum(heights_raw))
heights = heights_raw ./ sum(heights_raw)
println("доли после нормировки на 1 = ", heights)
plot(
    x, y,
    size = (700, 640),
    layout = grid(4, 1, heights = heights),
    xlabel = "x",
    ylabel = "y",
    legend = false,
    title = ["0.20" "0.30" "0.40" "0.15"],
```

```
)
saveplot("fig-5-46-heights.png")

p1 = plot(x, y, title = "Линии", xlabel = "x", ylabel = "y")
p2 = scatter(x, y, title = "Точки", xlabel = "x", ylabel = "y")
p3 = plot(
    x, y[:, 1:2],
    xlabel = "x",
    ylabel = "y",
    lw = 2,
    title = "Две колонки",
    label = ["колонка 1" "колонка 2"],
)
p4 = histogram(y, title = "Гистограммы столбцов", xlabel = "значение", ylabel = "част
plot(
    p1, p2, p3, p4,
    layout = (2, 2),
    legend = false,
    size = (800, 600),
    background_color = :ivory,
)
saveplot("fig-5-47-combine.png")

seriestypes = [:step, :sticks, :bar, :hline, :vline, :path]
titles = ["step" "sticks" "bar" "hline" "vline" "path"]
plot(
    rand(20, 1),
    seriestype = seriestypes,
```

```
    layout = (2, 3),
    ticks = nothing,
    legend = false,
    title = titles,
    markersize = 3,
    size = (800, 520),
)
saveplot("fig-5-48-kinds.png")

l = @layout [
    a{0.3w} [grid(3, 3)
             b{0.2h}]
]
plot(
    rand(10, 11),
    layout = l,
    legend = false,
    seriestype = [:bar :scatter :path],
    title = ["($i)" for j in 1:1, i in 1:11],
    titleloc = :right,
    titlefont = font(8),
    size = (900, 640),
)
saveplot("fig-5-49-layout.png")
```

```
сумма долей из методички = 1.05
доли после нормировки на 1 = [0.19047619047619047, 0.2857142857142857, 0.380952380952380
рисунок: fig-5-46-heights.png
рисунок: fig-5-47-combine.png
```

рисунок: fig-5-48-kinds.png

рисунок: fig-5-49-layout.png

"/home/glebq/2026-1--study--computer-practice/labs/lab05/report/image/fig-5-49-layout.png"

4.3 Задания для самостоятельной работы (раздел 5.4)

4.3.1 Задание 1. Типы графиков синуса

$y = \sin x$ на $[0, 2\pi]$. В одном окне собраны те типы серий, которыми эта кривая (или набор её значений) рисуется: линия, точки, столбцы, ступеньки, палочки, гистограмма значений, заливка, steppre и steppost.

```
xs = range(0, 2π, length = 120)
ys = sin.(xs)
xs_coarse = range(0, 2π, length = 24)
ys_coarse = sin.(xs_coarse)

panels = []

function try_panel(builder, name, panels; xlabel = "x", ylabel = "y")
    p = builder()
    plot!(p; title = name, legend = false, xlabel = xlabel, ylabel = ylabel)
    push!(panels, p)
    println("построен тип: ", name)
    return p
end
```

```
try_panel(() -> plot(xs, ys, color = :blue, lw = 2), "path", panels)
try_panel(() -> scatter(xs_coarse, ys_coarse, color = :blue), "scatter", panels)
try_panel(() -> bar(xs_coarse, ys_coarse, color = :steelblue), "bar", panels)
try_panel(() -> plot(xs, ys, seriestype = :step, color = :blue, lw = 2), "step", pane
try_panel(() -> plot(xs_coarse, ys_coarse, seriestype = :sticks, color = :blue), "sti
try_panel(
    () -> histogram(ys, color = :steelblue, bins = 15),
    "histogram",
    panels,
    xlabel = "sin(x)",
    ylabel = "частота",
)
try_panel(
    () -> plot(xs, ys, fillrange = 0, fillalpha = 0.45, color = :blue, lw = 2),
    "fill",
    panels,
)
try_panel(() -> plot(xs, ys, seriestype = :steppre, color = :blue), "steppre", panels
try_panel(() -> plot(xs, ys, seriestype = :steppost, color = :blue), "steppost", pane

rows = ceil(Int, length(panels) / 3)
plot(
    panels...,
    layout = (rows, 3),
    size = (960, 280 * rows),
    plot_title = "Типы графиков y = sin(x)",
)
saveplot("fig-task1-types.png")
```

```
построен тип: path
построен тип: scatter
построен тип: bar
построен тип: step
построен тип: sticks
построен тип: histogram
построен тип: fill
построен тип: steppre
построен тип: steppost
рисунок: fig-task1-types.png

"/home/glebq/2026-1--study--computer-practice/labs/lab05/report/image/fig-task1-types.png"
```

4.3.2 Задание 2. Оформление линий

У GR пять стилей линии: `solid`, `dash`, `dot`, `dashdot`, `dashdotdot`. Кривые сдвинуты по вертикали на 0,45, иначе они лежат друг на друге и виден только верхний стиль. Шестая кривая — сплошная с маркерами.

```
line_styles = [:solid, :dash, :dot, :dashdot, :dashdotdot]
xs = range(0, 2π, length = 500)
plt = plot(
    xlabel = "x",
    ylabel = "sin(x) со сдвигом",
    title = "Стили линий для y = sin(x)",
    legend = :topright,
    size = (800, 520),
```

```
)
for (i, style) in enumerate(line_styles)
    plot!(plt, xs, sin.(xs) .+ 0.45 * (i - 1), linestyle = style, lw = 2.4, label = s
end
xs_m = range(0, 2π, length = 18)
plot!(
    plt, xs_m, sin.(xs_m) .+ 0.45 * length(line_styles),
    linestyle = :solid,
    marker = :diamond,
    lw = 2,
    label = "solid + marker",
)
saveplot("fig-task2-lines.png")
```

```
рисунок: fig-task2-lines.png

"/home/glebq/2026-1--study--computer-practice/labs/lab05/report/image/fig-
task2-lines.png"
```

4.3.3 Задание 3. График с заданным оформлением

$$y(x) = \pi x^2 \ln x, \qquad x \in [0{,}2,\ 5].$$

Рамка зелёная, линия красная. Шрифт подписей — Times. Отступы `left_margin` и `bottom_margin` оставляют место подписям. Деления: по x шаг 0,5, по y шаг 20.

```
xs = range(0.2, 5, length = 400)
ys = π .* xs .^ 2 .* log.(xs)
```

```
ylo, yhi = extrema(ys)
println("задание 3: y от ", ylo, " до ", yhi)
y_ticks = (floor(Int, ylo / 20) * 20):(20):(ceil(Int, yhi / 20) * 20)
plot(
    xs, ys,
    color = :red,
    lw = 2.5,
    label = "y = π x² ln(x)",
    xlabel = "x",
    ylabel = "y",
    title = "График y = π x² ln(x)",
    foreground_color_border = :green,
    foreground_color_axis = :green,
    framestyle = :box,
    xticks = 0:0.5:5,
    yticks = y_ticks,
    guidefont = font("Times", 12),
    titlefont = font("Times", 14),
    tickfont = font("Times", 10),
    legendfont = font("Times", 11),
    left_margin = 12mm,
    bottom_margin = 10mm,
    top_margin = 6mm,
    right_margin = 6mm,
    size = (800, 520),
    legend = :topleft,
)
saveplot("fig-task3-log.png")
```

```
задание 3: y от -0.5778441403334981 до 126.40495805279656
рисунок: fig-task3-log.png
```

```
"/home/glebq/2026-1--study--computer-practice/labs/lab05/report/image/fig-task3-log.png"
```

4.3.4 Задание 4. Четыре окна для пяти точек

$x = (-2, -1, 0, 1, 2)$, $y = x^3 - 3x$. Точки, ломаная, ломаная с маркерами и гладкая кривая по густой сетке. Файл figure_besputin.png.

```
xv = [-2.0, -1.0, 0.0, 1.0, 2.0]
yv = xv .^ 3 .- 3 .* xv
xf = range(-2, 2, length = 200)
yf = xf .^ 3 .- 3 .* xf
println("задание 4: y(x) = ", yv)

p1 = scatter(xv, yv, title = "Точки", xlabel = "x", ylabel = "y", label = "y(x)", leg
p2 = plot(xv, yv, title = "Линии", xlabel = "x", ylabel = "y", label = "ломаная", leg
p3 = plot(xv, yv, title = "Линии и точки", xlabel = "x", ylabel = "y", label = "ломан
p4 = plot(xf, yf, title = "Кривая", xlabel = "x", ylabel = "y", label = "y = x³ - 3x"
plot(p1, p2, p3, p4, layout = (2, 2), size = (800, 700))
saveplot("figure_besputin.png")
```

```
задание 4: y(x) = [-2.0, 2.0, 0.0, -2.0, 2.0]
рисунок: figure_besputin.png
```

```
"/home/glebq/2026-1--study--computer-practice/labs/lab05/report/image/figure_besputin
```

4.3.5 Задание 5. Две функции и две оси

$x = 3, 3{,}1, \ldots, 6$, $y_1 = \pi x$, $y_2 = e^x \cos x$.

На общей оси размах y_2 на порядок больше размаха y_1, и прямая πx прижимается к полоске почти постоянной высоты. Вторая ось возвращает каждой кривой свой масштаб.

```
x = collect(range(3, 6, step = 0.1))
y1 = π .* x
y2 = exp.(x) .* cos.(x)
span1 = maximum(y1) - minimum(y1)
span2 = maximum(y2) - minimum(y2)
println("задание 5: y1 от ", minimum(y1), " до ", maximum(y1), ", размах ", span1)
println("задание 5: y2 от ", minimum(y2), " до ", maximum(y2), ", размах ", span2)
println("задание 5: размах y2 / размах y1 = ", span2 / span1)

plot(
    x, y1,
    label = "y1 = πx",
    xlabel = "x",
    ylabel = "y",
    title = "Общая ось ординат",
    legend = :topleft,
    grid = :on,
    color = :blue,
    lw = 2,
    marker = :circle,
    markersize = 3,
)
plot!(x, y2, label = "y2 = exp(x) cos(x)", color = :red, lw = 2)
saveplot("fig-task5-single.png")
```

```
plot(
    x, y1,
    label = "y1 = πx",
    xlabel = "x",
    ylabel = "y1 = πx",
    title = "Две оси ординат",
    legend = :left,
    grid = :on,
    color = :blue,
    lw = 2,
)
plot!(
    twinx(), x, y2,
    label = "y2 = exp(x) cos(x)",
    ylabel = "y2 = exp(x) cos(x)",
    color = :red,
    lw = 2,
    legend = :right,
    grid = :on,
)
saveplot("fig-task5-twin.png")
```

```
задание 5: y1 от 9.42477796076938 до 18.84955592153876, размах 9.42477796076938
задание 5: y2 от -35.86283371230767 до 387.36034029093076, размах 423.22317400323846
задание 5: размах y2 / размах y1 = 44.90537344910449
рисунок: fig-task5-single.png
рисунок: fig-task5-twin.png
"/home/glebq/2026-1--study--computer-practice/labs/lab05/report/image/fig-task5-twin.png"
```

4.3.6 Задание 6. Экспериментальные данные с погрешностью

Условный опыт: остывание раствора. Модель

$$T(t) = 22 + 65e^{-0{,}15t},$$

t — минуты от 0 до 20. Термометр даёт среднеквадратичную ошибку 0,8 °C, на графике она отложена усами. Зерно 42.

```
Random.seed!(42)
t_exp = collect(0:1:20)
□ = 0.8
model_T = 22 .+ 65 .* exp.(-0.15 .* t_exp)
measured = model_T .+ □ .* randn(length(t_exp))
println("задание 6, t = ", t_exp)
println("задание 6, измерение = ", measured)
println("задание 6, модель = ", model_T)
scatter(
    t_exp, measured,
    yerr = fill(□, length(t_exp)),
    xlabel = "время, мин",
    ylabel = "температура, °C",
    title = "Остывание раствора",
    label = "измерение, □ = 0.8 °C",
    legend = :topright,
    marker = :circle,
    color = :black,
    markersize = 5,
)
plot!(t_exp, model_T, label = "модель 22 + 65 exp(−0.15 t)", color = :red, lw = 2)
```

```
saveplot("fig-task6-cool.png")
```

```
задание 6, t = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20]
задание 6, измерение = [86.70931401483858, 78.14740824008814, 69.90119396737649, 63.196
задание 6, модель = [87.0, 77.94601846762876, 70.15318434431165, 63.44582985541527, 57.6
рисунок: fig-task6-cool.png
```

"/home/glebq/2026-1--study--computer-practice/labs/lab05/report/image/fig-task6-cool.png"

4.3.7 Задание 7. Точечный график случайных данных

```
Random.seed!(42)
n = 40
scatter(
    rand(n), rand(n),
    xlabel = "ось x",
    ylabel = "ось y",
    title = "Случайные точки на плоскости",
    label = "выборка из 40 точек",
    legend = :topright,
    marker = :circle,
    markersize = 6,
    color = :teal,
)
saveplot("fig-task7-scatter.png")
```

```
рисунок: fig-task7-scatter.png
```

"/home/glebq/2026-1--study--computer-practice/labs/lab05/report/image/fig-task7-scatter.png"

4.3.8 Задание 8. Трёхмерный точечный график

```
Random.seed!(42)
n = 50
scatter(
    rand(n), rand(n), rand(n),
    markersize = 4 .+ 10 .* rand(n),
    xlabel = "x",
    ylabel = "y",
    zlabel = "z",
    title = "Случайные точки в пространстве",
    label = "выборка из 50 точек",
    legend = :topright,
    color = :purple,
    alpha = 0.85,
    size = (720, 560),
)
saveplot("fig-task8-scatter3d.png")
```

рисунок: fig-task8-scatter3d.png

"/home/glebq/2026-1--study--computer-practice/labs/lab05/report/image/fig-task8-scatter3d.png"

4.3.9 Задание 9. Анимация синусоиды

На каждом кадре аргумент доходит дальше: от нуля до 2π. Три кадра сохранены отдельно, последовательность — в gif.

```
xs_full = range(0, 2π, length = 200)
nframes = 60

function sine_frame(i)
    m = max(2, round(Int, length(xs_full) * i / nframes))
    xs = xs_full[1:m]
    plot(
        xs, sin.(xs),
        xlim = (0, 2π),
        ylim = (-1.3, 1.3),
        xlabel = "x",
        ylabel = "sin(x)",
        title = "Построение синусоиды",
        label = "sin(x)",
        legend = :topright,
        lw = 2,
        color = :blue,
    )
end

for (i, name) in ((1, "fig-task9-start.png"), (30, "fig-task9-mid.png"), (nframes, "f
    sine_frame(i)
    saveplot(name)
end

anim = @animate for i in 1:nframes
    sine_frame(i)
end
```

```
gif(anim, joinpath(FIGDIR, "fig-task9-sine.gif"), fps = 12)
println("рисунок: fig-task9-sine.gif")
```

```
рисунок: fig-task9-start.png
рисунок: fig-task9-mid.png
рисунок: fig-task9-end.png
рисунок: fig-task9-sine.gif
```

```
[ Info: Saved animation to /home/glebq/2026-1--study--computer-practice/labs/lab05/report/image/fig-task9-sine.gif
```

4.3.10 Задания 10 и 11. Гипоциклоида и эпициклоида

Модуль $k = R/r$. Целые значения $k = 3$ и $k = 4$, рациональные $k = 5/2$ и $k = 7/3$. Для $k = p/q$ в несократимых целых кривая замыкается за q оборотов, поэтому параметр t идёт до $2\pi q$.

Эпициклоида — та же схема, окружность катится снаружи:

$$x = r(k+1)\cos t - r\cos((k+1)t), \quad y = r(k+1)\sin t - r\sin((k+1)t).$$

```
function k_turns(k)
    rq = k isa Rational ? k : rationalize(float(k); tol = 1e-8)
    return denominator(rq)
end

function rolling_frame(kind, k, t; r = 1.0)
    turns_title = kind == :hypo ? "Гипоциклоида" : "Эпициклоида"
    R = r * float(k)
    lim = kind == :hypo ? R + 0.45 : R + 2r + 0.45
```

```
    □ = range(0, 2π, length = 240)
    X = R .* cos.(□)
    Y = R .* sin.(□)
    plt = plot(
        xlim = (-lim, lim), ylim = (-lim, lim),
        aspect_ratio = 1, legend = false, framestyle = :origin,
        xlabel = "x", ylabel = "y",
        title = "$turns_title, k = $k",
        size = (640, 640),
    )
    plot!(plt, X, Y, color = :blue, lw = 1.5)
    if isempty(t)
        return plt
    end
    if kind == :hypo
        x = r * (k - 1) * cos.(t) .+ r * cos.((k - 1) .* t)
        y = r * (k - 1) * sin.(t) .- r * sin.((k - 1) .* t)
        cx = r * (k - 1) * cos(t[end])
        cy = r * (k - 1) * sin(t[end])
    else
        x = r * (k + 1) * cos.(t) .- r * cos.((k + 1) .* t)
        y = r * (k + 1) * sin.(t) .- r * sin.((k + 1) .* t)
        cx = r * (k + 1) * cos(t[end])
        cy = r * (k + 1) * sin(t[end])
    end
    plot!(plt, x, y, color = :red, lw = 2)
    □ = range(0, 2π, length = 80)
    plot!(plt, cx .+ r .* cos.(□), cy .+ r .* sin.(□), color = :black)
```

```
    plot!(plt, [cx, x[end]], [cy, y[end]], color = :black, marker = :circle, markersi
    scatter!(plt, [x[end]], [y[end]], color = :red, markerstrokecolor = :red, markers
    return plt
end

function animate_roll(kind, k; r = 1.0, frames_per_turn = 70)
    turns = k_turns(k)
    nframes = frames_per_turn * turns
    □ = collect(range(0, 2π * turns, length = nframes))
    tag = kind == :hypo ? "hypo" : "epi"
    rq = k isa Rational ? k : rationalize(float(k); tol = 1e-8)
    slug = denominator(rq) == 1 ? string(numerator(rq)) : "$(numerator(rq))_$(denomin
    anim = @animate for i in 1:nframes
        rolling_frame(kind, k, □[1:i]; r = r)
    end
    gifname = "fig-$(tag)-$(slug).gif"
    pngname = "fig-$(tag)-$(slug).png"
    gif(anim, joinpath(FIGDIR, gifname), fps = 15)
    rolling_frame(kind, k, □; r = r)
    saveplot(pngname)
    println("анимация $tag, k = $k, оборотов = $turns, файл $gifname")
end

for k in (3, 4, 5 // 2, 7 // 3)
    animate_roll(:hypo, k)
end

for k in (3, 4, 5 // 2, 7 // 3)
```

```
    animate_roll(:epi, k)
end

println("Готово. Рисунки: ", FIGDIR)
```

```
рисунок: fig-hypo-3.png
анимация hypo, k = 3, оборотов = 1, файл fig-hypo-3.gif

[ Info: Saved animation to /home/glebq/2026-1--study--computer-practice/labs/lab05/report/image/fig-hypo-3.gif

рисунок: fig-hypo-4.png
анимация hypo, k = 4, оборотов = 1, файл fig-hypo-4.gif

[ Info: Saved animation to /home/glebq/2026-1--study--computer-practice/labs/lab05/report/image/fig-hypo-4.gif

рисунок: fig-hypo-5_2.png
анимация hypo, k = 5//2, оборотов = 2, файл fig-hypo-5_2.gif

[ Info: Saved animation to /home/glebq/2026-1--study--computer-practice/labs/lab05/report/image/fig-hypo-5_2.gif

рисунок: fig-hypo-7_3.png

[ Info: Saved animation to /home/glebq/2026-1--study--computer-practice/labs/lab05/report/image/fig-hypo-7_3.gif


анимация hypo, k = 7//3, оборотов = 3, файл fig-hypo-7_3.gif
рисунок: fig-epi-3.png
анимация epi, k = 3, оборотов = 1, файл fig-epi-3.gif
```

```
[ Info: Saved animation to /home/glebq/2026-1--study--computer-
practice/labs/lab05/report/image/fig-epi-3.gif

рисунок: fig-epi-4.png
анимация epi, k = 4, оборотов = 1, файл fig-epi-4.gif

[ Info: Saved animation to /home/glebq/2026-1--study--computer-
practice/labs/lab05/report/image/fig-epi-4.gif

рисунок: fig-epi-5_2.png
анимация epi, k = 5//2, оборотов = 2, файл fig-epi-5_2.gif

[ Info: Saved animation to /home/glebq/2026-1--study--computer-
practice/labs/lab05/report/image/fig-epi-5_2.gif

рисунок: fig-epi-7_3.png
анимация epi, k = 7//3, оборотов = 3, файл fig-epi-7_3.gif
Готово. Рисунки: /home/glebq/2026-1--study--computer-practice/labs/lab05/report/image

[ Info: Saved animation to /home/glebq/2026-1--study--computer-
practice/labs/lab05/report/image/fig-epi-7_3.gif
```

5 Результаты

5.1 Повторение примеров из раздела 5.2

1. Одна кривая $f(x) = (3x^2 + 6x - 9)\, e^{-0{,}3x}$ на $[-5, 10]$ построена бэкендами GR, PyPlot, plotly и UnicodePlots. При $x = -5$ значение около 161, минимум около $-17{,}7$ при $x \approx -1{,}5$.
2. Пять слагаемых ряда Тейлора держатся рядом с $\sin x$ на $[-\pi, \pi]$ и расходятся к краям $[-2\pi, 2\pi]$. На оформленном графике шаг делений по x равен 1, по y равен 0,5, шрифт делений — Times, подписи оси x повёрнуты на 45°.
3. Точечные графики построены с зерном 42. Для десяти точек y равен 0,174; 0,322; 0,259; 0,166; 0,527; 0,483; 0,391; 0,803; 0,722; 0,372.
4. Полином 5-й степени для $e^{1+x} + 4000\varepsilon$ дал коэффициенты $-587{,}66$; 2329,56; $-2518{,}17$; 935,64; $-141{,}44$; 7,66. Норма невязки 125163, то есть на уровне шума $4000\sqrt{1000}$.
5. `twinx`, полярные и параметрические кривые, поверхности, линии уровня и градиент построены отдельными вызовами `plot`. Доли высоты подграфиков в методичке суммируются в 1,05; эта версия `Plots` принимает только сумму 1, поэтому доли поделены на 1,05.
6. Шесть серий погрешностей, $n = 10$. Полуширины $1{,}96\,\sigma/\sqrt{n}$ равны 0,620; 0,310; 0,155; 0,077; 0,039; 0,019. Смесь трёх нормальных распределений содержит 1834 числа, среднее 32,55.

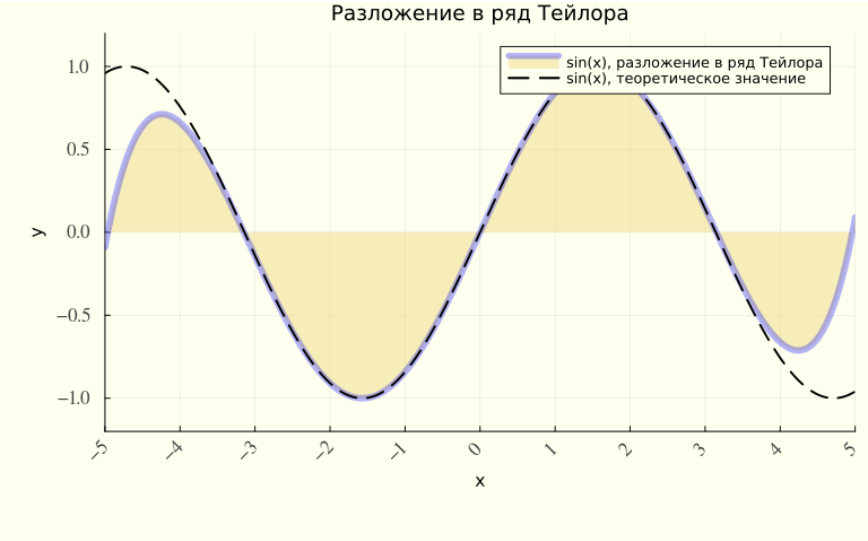


Рисунок 5.1: sin(x) и частичная сумма ряда Тейлора

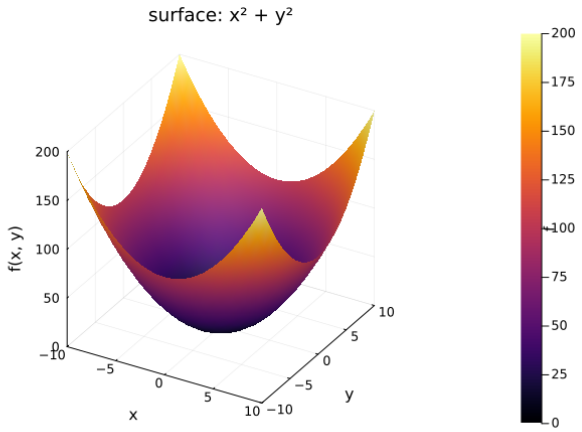


Рисунок 5.2: Поверхность x² + y²

5.2 Задания для самостоятельной работы

1. На $[0, 2\pi]$ в одном окне собраны девять представлений $y = \sin x$: линия, точки, столбцы, ступенька, палочки, гистограмма значений, заливка до нуля, `steppre` и `steppost`.
2. У GR пять стилей линии. Шестая кривая — сплошная с маркерами. Каж-

дая следующая поднята на 0,45.

3. На $[0{,}2,\ 5]$ функция $\pi x^2 \ln x$ меняется от $-0{,}578$ до $126{,}4$. Линия красная, рамка зелёная, шрифт Times, шаг по x равен 0,5, по y — 20.
4. Для $x = (-2, -1, 0, 1, 2)$ значения $y = x^3 - 3x$ равны $(-2, 2, 0, -2, 2)$. Четыре окна сохранены в `figure_besputin.png`.
5. На $x = 3; 3{,}1; \ldots; 6$ размах $y_1 = \pi x$ равен 9,42, размах $y_2 = e^x \cos x$ равен 423. Отношение размахов 44,9, поэтому на общей оси прямая почти не видна, а `twinx` возвращает каждой кривой свою шкалу.
6. Остывание задано моделью $T(t) = 22 + 65e^{-0{,}15t}$. В нуле измерение 86,7 при модели 87,0, на 10-й минуте 35,8 при модели 36,5, на 20-й — 24,5 при модели 25,2.
7. Построены плоский и пространственный точечные графики и анимация синусоиды на неподвижном окне $[0, 2\pi]$.
8. Гипоциклоида и эпициклоида построены для $k = 3$, $k = 4$, $k = 5/2$ и $k = 7/3$. Для рационального $k = p/q$ параметр делает q оборотов, иначе кривая не замыкается.

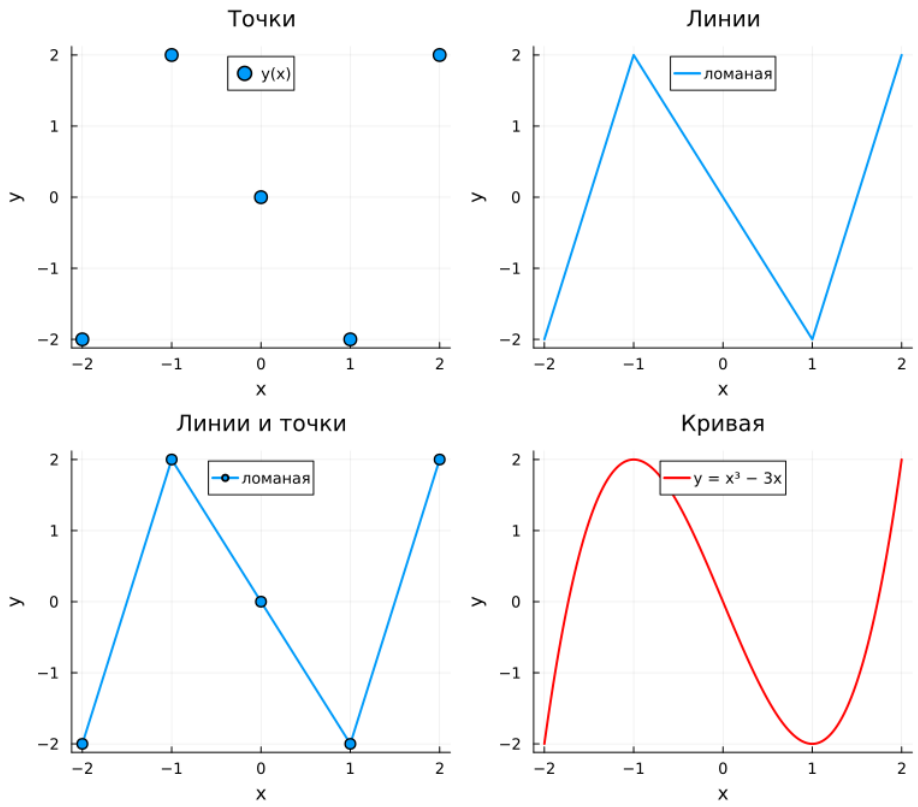


Рисунок 5.3: Четыре окна для y = x³ − 3x

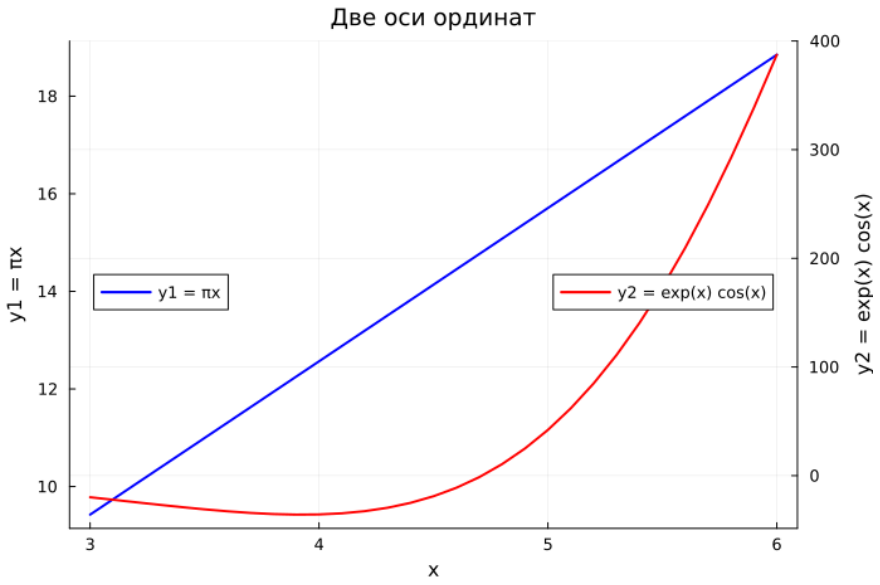


Рисунок 5.4: πx и e cos x на двух осях

6 Выводы

1. Один вызов `plot` в `Plots` рисуется бэкендами GR, PyPlot, plotly и UnicodePlots. Русские подписи проходят и в GR, и в PyPlot. plotly дополнительно отдаёт html, UnicodePlots — текстовый график.

2. Частичная сумма ряда Тейлора до степени 9 совпадает с $\sin x$ вблизи нуля и отходит к краям отрезка $[-2\pi, 2\pi]$. Заливка, шаг делений, поворот подписей и цвет фона задаются атрибутами той же команды `plot`.

3. Размер маркера переносит третье число на плоский точечный график. `scatter` с тремя массивами строит точки в пространстве.

4. Коэффициенты полинома 5-й степени считаются оператором `\`. На шуме 4000ε невязка остаётся порядка нормы этого шума: полином повторяет ход экспоненты.

5. Если размахи рядов отличаются в десятки раз, общая ось скрывает меньший ряд. Для πx и $e^x \cos x$ отношение размахов равно 44,9. `twinx()` возвращает каждому ряду свою шкалу.

6. `contour` и `quiver!` показывают линии уровня и градиент. `@animate` и `gif` собирают поверхность, синусоиду и циклоиды. Для $k = p/q$ параметр нужно вести q оборотов, иначе кривая не замыкается.

7. Усы `yerr` и лента `ribbon` рисуют одну и ту же погрешность $1{,}96\,\sigma/\sqrt{n}$. Смесь трёх нормальных распределений на гистограмме даёт три горба.

Сетка окон задаётся `layout`, `grid` и `@layout`. Доли высоты в этой версии `Plots` обязаны давать в сумме 1.

Список литературы

1. Julia Documentation. — URL: https://docs.julialang.org/en/v1/.
2. Plots.jl. — URL: https://docs.juliaplots.org/stable/.
3. Distributions.jl. — URL: https://juliastats.org/Distributions.jl/stable/.
4. Klok H., Nazarathy Y. Statistics with Julia: Fundamentals for Data Science, Machine Learning and Artificial Intelligence. — 2020. — URL: https://statisticswithjulia.org/.
5. Кулябов Д. С. и др. Компьютерный практикум на языке Julia. Лабораторная работа № 5. Построение графиков.